

Modern Large Language Models in Generative Design

A. V. Guryanov
Ulyanovsk State Technical University
Ulyanovsk, Russia
aleksey.guryanov.2012@ya.ru

V. S. Moshkin
Ulyanovsk State Technical University
Ulyanovsk, Russia
v.moshkin@ulstu.ru

Abstract—This article compares four large language models in the task of generative design. The models compared are Qwen3VL-30B-Thinking, Qwen3.5-35B, GPT-5.2, and DeepSeek-V3.1-671B. The comparison is conducted based on the results of five experiments involving the generation of five different CAD models from text descriptions and images using the OpenSCAD scripting language for CAD modeling. Based on the experimental results, it was concluded that models with vision capabilities perform better in this task than purely text-based models. The obtained results indicate that there are grounds to suggest that models with vision capabilities possess better spatial reasoning. Additionally, it was found that the Qwen model family's vision capabilities exhibited rapid growth between their first and second iterations.

Keywords—large language models, generative design, design automation, CAD modeling, OpenSCAD

I. INTRODUCTION

The creation of production artifacts (such as CAD part models, semantic structures, action sequences, instructions, drawings, etc.) is an extremely time- and labor-intensive process. Automating this process using classical algorithmic methods is practically impossible due to its complexity and multidimensional nature.

Modern generative artificial intelligence models such as GPT, Claude, Grok, Gemini, DeepSeek, GLM, and Qwen are developing at a rapid pace and already demonstrate the ability to solve a wide range of tasks that are beyond the capabilities of classical algorithms. They have significant potential in the field of production automation, particularly at the design stage.

Generative design is an iterative process of interaction between a human and specialized software aimed at simplifying and accelerating the creation of production artifacts. Generative design is mainly used to automate and accelerate the creation of CAD models of non-standard engineering objects with simple to moderately complex geometry. It can also be used for rapid creation and prototyping in applied 3D printing, as well as for structural optimization.

Early research on generative design using large language models began shortly after the emergence of the first practically applicable LLMs in the late 2010s and early 2020s. However, meaningful results began to appear only after models at the level of GPT-4 emerged in 2023. The development and spread of multimodal models, as well as approaches such as chain-of-

thought reasoning and agent workflows, gave new momentum to this research area in 2024-2025.

This work provides a preliminary evaluation of several currently relevant large language models in the task of generative design.

II. REVIEW OF PREVIOUS WORKS ON GENERATIVE DESIGN

Generative design has been examined in various studies.

In total, 16 papers dedicated to the generation of production artifacts using large language models were analyzed. The reviewed studies fall into three categories:

- Code Generation
- Multimodal Systems
- Tool Integration

Category #1 (Code Generation) [1-6] includes works devoted to exclusively text-based large language models.

Advantages:

- Rapid prototyping using familiar languages (Python, CadQuery, OpenSCAD).
- Direct traceability of output – generated code can be read and manually edited.
- Works well for logical and textual tasks and chain-of-thought reasoning.
- Lower computational complexity compared to multimodal models.
- Easier integration into continuous integration/deployment methodologies and automated tests.

Disadvantages:

- Limited understanding of visual/spatial information without additional measures.
- Frequent syntax and logic errors when describing complex geometry – execution and validation are required.
- Fewer capabilities for perceiving images or 3D data out of the box.

- Requires a rich execution environment and test generation to verify validity.

Category #2 (Multimodal Systems) [7-13] includes works dedicated to multimodal models capable of processing not only text but also other data types.

Advantages:

- Native support for images, point clouds, and visual representations.
- Better ability to connect visual details with textual instructions.
- Potential for a unified multimodal representation, simplifying alignment between “what I see” and “what I generate.”
- Support for spatial reasoning and specialized tokens for 3D.
- More convenient for tasks requiring visual verification and interactivity.

Disadvantages:

- Higher computational and model complexity — more expensive to train and deploy.
- Harder to interpret internal representations and verify output validity.
- Risk of degraded text generation quality if modalities are poorly integrated.
- Require large multi-domain datasets and complex preprocessing.
- Fine-tuning modality alignment introduces additional sources of error.

Category #3 (Tool Integration) [14-16] includes works focused on integrating LLMs into existing CAD systems.

Advantages:

- Direct path to practical value (automation and reduced manual work).
- Ability to combine strengths of different models (agents, executors, validation).
- Easier validation using existing CAD APIs, simulators, and tests.
- Support for checkpoints and quality monitoring in production.
- Enables hybrid workflows (human + model).

Disadvantages:

- High integration complexity (security, API versions, access management).
- Need for reliable validation and rollback mechanisms in automated actions.
- Dependence on tool ecosystems (compatibility, licensing).

- May require significant infrastructure and maintenance engineering.
- Harder to scale model changes without affecting production processes.

Most of the reviewed works explored model capabilities after fine-tuning or other modifications. The goal of this work is instead to study the capabilities of models in their original form in order to estimate their potential for further development and to determine the impact of vision capabilities on spatial reasoning quality.

III. DESCRIPTION OF THE MODELS AND EXPERIMENTAL SETUP

A total of three large language models were evaluated.

The first model is Qwen3VL-30B-Thinking, developed by Alibaba and released in September 2025. It showed strong results in benchmarks compared to other open-weight models of its time. All variants of this model (from 2 to 235 billion parameters) include built-in vision capabilities, a first for its family, and have demonstrated particularly strong performance on tasks requiring such capabilities for the model’s class, sizes, and release date. The model exists in instruct and reasoning versions; the reasoning version was chosen for the experiments as it is more capable, though significantly slower. It uses a Mixture-of-Experts (MoE) architecture, which allows relatively fast inference even when some weights are partially loaded into RAM rather than GPU memory. With 30 billion parameters and Q4_K_M quantization, the model can run on mid-range consumer hardware (12 GB VRAM and 32 GB RAM) with acceptable speed. Combined with its vision capabilities, this accessibility motivated its selection. The model was run locally using LM Studio.

The second model is Qwen3.5-35B, a continuation of Alibaba’s Qwen family released in February 2026. Compared to Qwen3VL, Qwen3.5 is an iterative improvement, sharing much of its architecture with its predecessor. It achieves noticeably better performance through more robust training techniques, including much earlier introduction of multimodal tokens. Notably, the model is no longer split into separate instruct and reasoning versions – it is a unified model in which reasoning is a runtime toggle. This model was selected to evaluate how much of an improvement a more refined second iteration of vision capabilities within the same family of models could be, considering the general tendency of brand-new advancements to follow logarithmic growth that is rapid in the early stages. Because this model as well as the previous one are much smaller than the others, minor output errors were corrected and ignored during evaluation. Such errors include small mistakes that even someone unfamiliar with CAD modeling could easily fix, such as redundant constructs or misplaced variables in otherwise correct code.

The third model is GPT-5.2, a proprietary model from OpenAI released in December 2025 and considered one of the most capable existing models. It also exists in instruct and reasoning variants; the instruct version was chosen to avoid giving the already much stronger model an additional advantage. This model served as the reference baseline among modern

LLMs. It was accessed via the standard chatgpt.com web interface.

The fourth model is DeepSeek-V3.1-671B, released in August 2025. One of its key features compared to earlier DeepSeek models is the integration of both reasoning and instruct modes in a single model. The reasoning mode was used for the purposes of this work. With 671 billion parameters, it is one of the largest open-weight models available to ordinary users through free cloud services. However, it does not possess vision capabilities. These two factors were the main reasons this model was chosen – the primary interests here were investigating the importance of vision capabilities and determining whether a model's massive size could compensate for its absence of vision capabilities. It was run through Ollama in a cloud environment provided for free by the program.

The OpenSCAD language was chosen as the format for describing CAD models. OpenSCAD is a script-based CAD language that describes models using geometric primitives and operations such as unions and differences. Unlike direct 3D representations defined through vertices and face coordinates, OpenSCAD is structurally closer to programming languages, easier to read, and more compact. These characteristics make OpenSCAD particularly well-suited for use by LLMs, which traditionally struggle with precise spatial reasoning.

Five experiments were conducted with each model to test their ability to generate CAD models in OpenSCAD from text descriptions.

The first experiment involved the generation of a simple five-wall box container with a hollow interior. This is one of the simplest geometric shapes useful in everyday life. If the model cannot generate this shape, it is highly unlikely that it will be able to generate anything more geometrically complex. The prompt in this experiment was: «Write an OpenSCAD script that describes a simple box container. Make the width, height, depth, and wall thickness parametric».

The second experiment involved the generation of a simple L-shaped bracket. This item also has many uses, but its geometric description is somewhat more complex. The prompt in this experiment was: «Write an OpenSCAD script that describes a simple L-shaped bracket. Make its length and height parametric».

The third experiment involved the generation of a simple 4-way pipe connector. This part is also not particularly complex, but its design includes shapes other than prisms, namely cylinders. This experiment also included an image of the desired part, and for models with vision capabilities, tested their ability to connect input images with the desired result – OpenSCAD code. The prompt in this experiment is: «Write an OpenSCAD script that describes a simple 4-way pipe connector (see image). Make the diameter of the hole and the thickness of the pipe parametric».

The fourth experiment involved the generation a hollow vase. This experiment did not strictly specify the desired shape and tested not only the models' ability to generate more complex shapes, but also their creativity. The prompt in this experiment was: «Write an OpenSCAD script that describes a hollow vase with one opening at the top».

The fifth experiment tested the models' ability to edit existing OpenSCAD code. The models were given an existing OpenSCAD script describing a box container with rounded corners and a lid found on the Internet [17] and asked to remove the rounded corners of the container, making them straight and sharp. The particular challenge here is that the rounded corners are implemented using cylinders; simply setting their radius to zero makes the model invalid, requiring a more sophisticated modification. In this experiment, models with vision capabilities were also provided with an image of the box container in its original form, with rounded corners. The prompt in this experiment was: «Below is an OpenSCAD script that describes a simple box with rounded corners (see attached image). Edit the script so that the corners are not rounded but instead sharp. *OpenSCAD script describing the container*».

IV. EXPERIMENT RESULTS AND THEIR EVALUATION

The results of experiments on the Qwen3VL-30B-Thinking model are presented in Table 1. This model successfully completed experiments #1-#3, partially completed experiment #5 (two of the four walls are slightly thicker than the other two, but overall the task was completed), but was unable to generate a valid vase model in experiment #4. The satisfactory performance on clearly defined tasks suggests good potential of the model, while its failure in a more creative task and the need for minor but still necessary code corrections in Experiments #2, #3, and #5 likely result from the model's relatively small size.

The results of experiments on the Qwen3.5-35B model are presented in Table 2. This model successfully completed Experiments #1, #2, #4, #5 and partially completed Experiment #3. In that experiment, the model created an additional pipe, making a 6-way connector instead of a 4-way one. However, this is not a major mistake, as it is easily correctable.

The results of experiments on the GPT-5.2 model are presented in Table 3. This model successfully completed Experiments #1, #2, and #5, and partially completed Experiments #3 and #4. Much like Qwen3.5-35B, in Experiment #3, the model created an additional pipe. The vase generated in Experiment #4, although had a rather unusual shape and high center of gravity, technically satisfied the prompt and, in theory, can be used for its intended purpose.

The results of experiments on the Deepseek-V3.1-671B model are presented in Table 4. This model successfully completed Experiments #1, #2, and #5, but failed experiments #3 and #4. In Experiment #3, it generated a chaotic structure instead of a four-way connector, while the vase generated in Experiment #4 contained a central rod and had a base in the shape of an uncut cone, making it unable to stand and therefore completely unsuitable for its intended use.

Based on the results of the conducted experiments, it can be concluded that models with vision capabilities do indeed possess better spatial reasoning and perform noticeably better in CAD model generation tasks compared to purely text-based models, even when the latter have a significant advantage in the number of parameters. The Qwen3VL-30B-Thinking model, which has vision capabilities, despite being more than 20 times smaller than the DeepSeek-V3.1-671B model that lacks them, demonstrated comparatively much better results. Both models

TABLE I. RESULTS OF EXPERIMENTS ON THE QWEN3VL-30B-THINKING MODEL

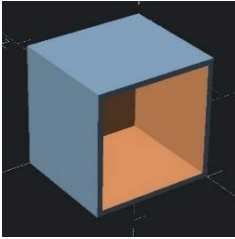
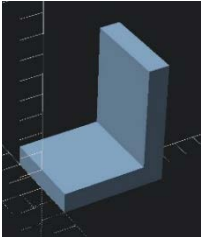
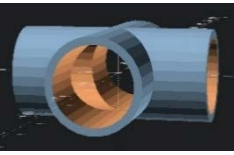
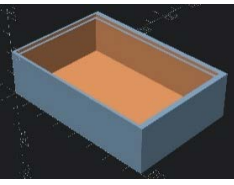
Experiment #1 – Generation of a simple box container	Experiment #2 – Generation of a simple L-shaped bracket	Experiment #3 – Generation of a simple 4-way pipe connector	Experiment #4 – Generation of a hollow vase	Experiment #5 – Existing code editing
			Model is invalid	

TABLE II. RESULTS OF EXPERIMENTS ON THE QWEN3.5-35B MODEL

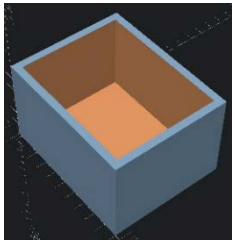
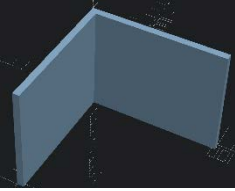
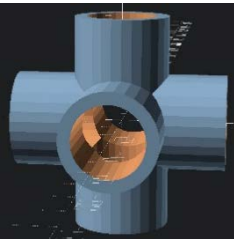
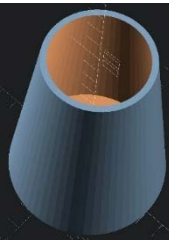
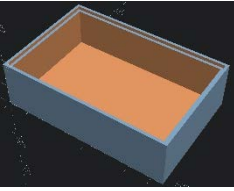
Experiment #1 – Generation of a simple box container	Experiment #2 – Generation of a simple L-shaped bracket	Experiment #3 – Generation of a simple 4-way pipe connector	Experiment #4 – Generation of a hollow vase	Experiment #5 – Existing code editing
				

TABLE III. RESULTS OF EXPERIMENTS ON THE GPT-5.2 MODEL

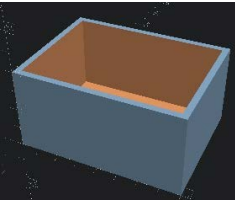
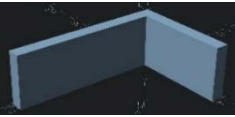
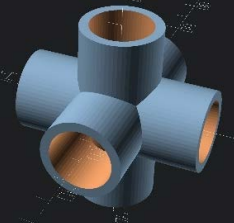
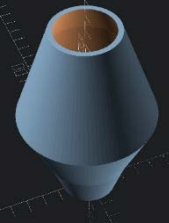
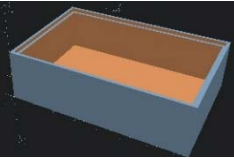
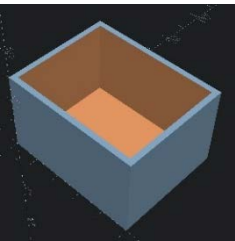
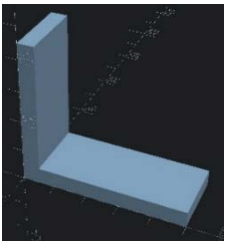
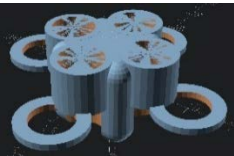
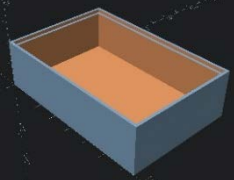
Experiment #1 – Generation of a simple box container	Experiment #2 – Generation of a simple L-shaped bracket	Experiment #3 – Generation of a simple 4-way pipe connector	Experiment #4 – Generation of a hollow vase	Experiment #5 – Existing code editing
				

TABLE IV. RESULTS OF EXPERIMENTS ON THE DEEPSEEK-V3.1-671B MODEL

Experiment #1 – Generation of a simple box container	Experiment #2 – Generation of a simple L-shaped bracket	Experiment #3 – Generation of a simple 4-way pipe connector	Experiment #4 – Generation of a hollow vase	Experiment #5 – Existing code editing
				

failed Experiment #4; however, in Experiment #3 – where DeepSeek-V3.1-671B failed completely and did not produce anything even remotely resembling the desired result – Qwen3VL-30B-Thinking made only non-fundamental errors that were easy to correct. Qwen3.5-35B proved to be an overall improvement over Qwen3VL-30B-Thinking: not only did it complete Experiment #4, which its predecessor failed, it even outperformed the much more capable GPT-5.2 by creating a more functional vase. It also avoided the mistake made by its predecessor in Experiment #5 and required far fewer, smaller corrections.

CONCLUSION

A preliminary evaluation of several modern large language models in the task of generative design was conducted. The experiments demonstrated that models with vision capabilities perform better in generative design tasks than purely text-based models, and that even the second iteration of vision capabilities within the same family of models can already provide significantly better results than the first. These results indicate that there are grounds that allow us to suggest with a certain degree of confidence that models with vision capabilities possess better spatial reasoning. Future research will primarily focus on open-weight multimodal models, as they offer greater potential compared to purely text-based models. Unlike proprietary models, they can run fully locally on the user's PC/datacenter, eliminating the risk of confidential data leakage and allowing use in domains with any level of secrecy. Furthermore, open-weight models can be modified or fine-tuned, making them more flexible. Future work will investigate the ability of models to generate not only CAD files but also other production artifacts, such as instructions.

ACKNOWLEDGMENT

This study was supported by the Ministry of Science and Higher Education of Russia in framework of project No. 075-03-2026-524 "Research and development of intelligent models, methods and algorithms for solving control problems under uncertainty based on knowledge engineering, machine learning and explainable intelligence".

REFERENCES

- [1] A. Daareyni, A. Martikkala, and H. Mokhtarian, "Generative AI meets CAD: Enhancing engineering design to manufacturing processes with large language models," *International Journal of Advanced Manufacturing Technology*, vol. 5, pp. 1893–1902, Aug. 2025, doi: 10.1007/s00170-025-15830-2.
- [2] A. Schüpbach, R.S. Miguel, J. Ferchow, and M. Meboldt, "From text to design: A framework to leverage LLM agents for automated CAD generation," *Proceedings of the Design Society*, 2025, doi: doi:10.1017/pds.2025.10203.
- [3] Y. Guan, X. Wang, X. Ming, J. Zhang, D. Xu, and Q. Yu, "CAD-Coder: Text-to-CAD generation with Chain-of-Thought and geometric reward," 2025, arXiv: 2505.19713.
- [4] B. E. Hizmi, Y. Sterman, and G. Austern, "LLMto3D - Generation of parametric, 3D printable objects using large language models," *International Journal of Architectural Computing*, vol. 23, no. 3, pp. 701–719, Jun. 2025, doi: 10.1177/14780771251353792.
- [5] Y. Du, S. Chen, W. Zan, P. Li, M. Wang, D. Song, B. Li, Y. Hu, and B. Wang, "BlenderLLM: Training large language models for computer-aided design with self-improvement," 2024, arXiv: 2412.14203.
- [6] S. Kumar, S. Kapoor, H. Vardhan, and Y. Zhao, "Generative AI for CAD automation: Leveraging large language models for 3D modelling," 2025, arXiv: 2508.00843.
- [7] S. Wu, A. Khasahmadi, M. Katz, P. K. Jayaraman, Y. Pu, K. Willis, and B. Liu, "CAD-LLM: Large language model for CAD," *NeurIPS 2023 Workshop on Machine Learning for Creativity and Design*, 2023.
- [8] S. Wu, A. Khasahmadi, M. Katz, P. K. Jayaraman, Y. Pu, K. Willis, and B. Liu, "CadVLM: Bridging language and vision in the generation of parametric CAD sketches," 2024, arXiv: 2409.17457.
- [9] J. Xu, C. Wang, Z. Zhao, W. Liu, Y. Ma, and S. Gao, "CAD-MLLM: Unifying multimodality-conditioned CAD generation with MLLM," 2024, arXiv: 2411.04954.
- [10] S. Wang, C. Chen, X. Le, Q. Xu, L. Xu, Y. Zhang, and J. Yang, "CAD-GPT: Synthesising CAD construction sequence with spatial reasoning-enhanced multimodal LLMs," 2024, arXiv: 2412.19663.
- [11] K. Niu, H. Yu, Z. Chen, M. Zhao, T. Fu, B. Li, and X. Xue, "From intent to execution: Multimodal Chain-of-Thought reinforcement learning for precise CAD code generation," 2025, arXiv: 2508.10118.
- [12] X. Li, Y. Sun, and Z. Sha, "LLM4CAD: Multi-modal large language models for three-dimensional computer-aided design generation," *Journal of Computing and Information Science in Engineering*, vol. 25, no. 2, pp. 1–16, 2024, doi: 10.1115/1.4067085.
- [13] X. Li, J. Li, Y. Song, Y. Lou, and X. Zhou, "Seek-CAD: A self-refined generative modeling for 3D parametric CAD using local inference via DeepSeek," 2025, arXiv: 2505.17702.
- [14] D. Murarka and D. P. Ghosh, "Conversational CAD: A natural language interface for parametric 3D modeling," *ResearchGate*, Jul. 2025, doi: 10.13140/RG.2.2.13587.18724.
- [15] L. Xiaorui, Z. Yuhao, W. Leiqi, G. Lexiang, X. Yaning, Z. Ke, C. Qianqian, and Z. Gang, "LLMs driven fusion AI-AD system for mechanical design: From understanding to generation," *Advanced Engineering Informatics*, vol. 68, part C, 2025, doi: 10.1016/j.aei.2025.103745.
- [16] H. Deng, S. Khan, and J.A. Erkoyuncu, "An investigation on utilizing large language model for industrial computer-aided design automation," *Procedia CIRP*, vol. 128, pp. 221–226, 2024, doi: 10.1016/j.procir.2024.07.049.
- [17] Fierman, "Simple lidded box," 2017, Thingiverse. [Online]. Available: <https://www.thingiverse.com/thing:2516757>